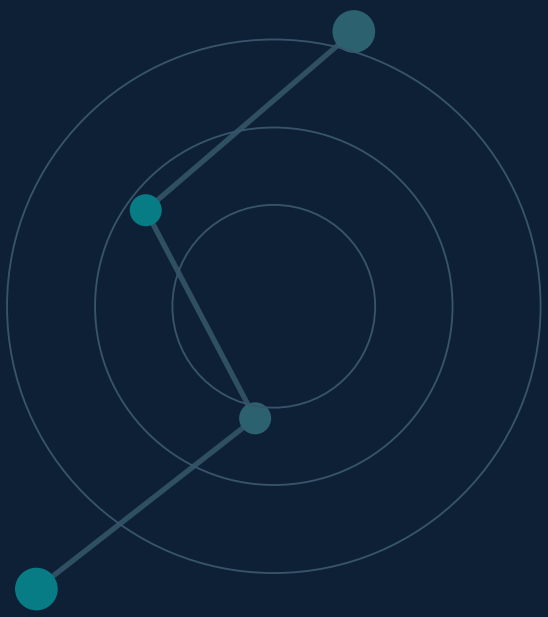
CAMPUSNAV3D / GUÍA DE EJECUCIÓN

Construye, comprende y defiende tu TFG

Manual de trabajo paso a paso para Carlos



16 módulos · 4 hitos · aprendizaje con IA

Pruebas, investigación y preparación de la defensa

Edición de trabajo | 8 de noviembre de 2026

Basada en la guía del tutor y el repositorio CampusNav3D

MANUAL PARA CARLOS

Tu proyecto, bajo tu control

Hola, Carlos. Esta guía está pensada para ayudarte a **retomar el control intelectual de CampusNav3D** sin renunciar a la velocidad que ofrece la IA. No necesitas tirar el repositorio existente ni aprender cinco tecnologías completas antes de comenzar: necesitas un mapa, pequeños entregables verificables y la capacidad de explicar tus decisiones.

El objetivo del TFG no es memorizar miles de líneas de código, sino demostrar que comprendes el problema, sabes construir una solución, verificarla, reconocer sus límites y defender por qué la has diseñado así.

> **Regla central:** reutiliza todo lo que puedas justificar y probar. Corrige o reemplaza únicamente aquello que no funcione, contradiga los requisitos o esté peor diseñado que una alternativa que sepas defender.

Cómo utilizar este manual

01. Lee primero la ruta de cuatro hitos y el protocolo de trabajo con IA. No intentes aprender los 16 módulos a la vez.
02. En cada módulo, parte del código real: verifica el estado, estudia los conceptos necesarios y construye un resultado pequeño.
03. Antes de cerrar una tarea, ejecuta pruebas, documenta decisiones y explica el funcionamiento con tus propias palabras.
04. Cada semana revisa avances, bloqueos y el siguiente paso con tu mentor; el tutor académico sigue marcando los criterios de evaluación.
05. Prepara la evaluación experimental y la memoria desde el principio. Una demo funcional no sustituye a un estudio científico.

De dónde sale esta propuesta

El mapa de trabajo se basa en la **Guía de implementación y evaluación para convertir el TFG en un artículo publicable en SoftwareX** (versión 1.0, junio de 2026) y en la inspección de los archivos de la rama principal de DiZaster2002/CampusNav3D. Es una **propuesta operativa complementaria**, no una modificación de los requisitos de tu tutor.

Los estados del repositorio son una fotografía orientativa de la revisión realizada en noviembre de 2026: **no se han ejecutado localmente todas las pruebas** ni verificado el sistema de extremo a extremo. Debes comprobarlos antes de dar nada por hecho.

Mapa de navegación del manual

Tu proyecto, bajo tu control . . . 1
Mapa de navegación del manual . . . 2
01 · El proyecto en cuatro hitos . . . 3
02 · Tu punto de partida: seis sesiones . . . 4
03 · Protocolo de trabajo con IA . . . 5
04 · Los 16 módulos de CampusNav3D . . . 6
M01 · Entorno e infraestructura de desarrollo . . . 7
M02 · Modelo de dominio y persistencia espacial . . . 8
M03 · Importación y preprocesamiento de planos . . . 9
M04 · Extractores procedimentales y proveedores de IA . . . 10
M05 · Normalización, geometrías y validación espacial . . . 11
M06 · Corrección humana, estados y versionado . . . 12
M07 · Generación y validación del grafo de navegación . . . 13
M08 · Motor de rutas y perfiles de accesibilidad . . . 14
M09 · API, orquestación y trabajos asíncronos . . . 15
M10 · Aplicación web y mapa 2D . . . 16
M11 · Generación y visualización 3D . . . 17
M12 · Sincronización entre 2D, 3D y navegación . . . 18
M13 · Calidad, pruebas, seguridad y accesibilidad . . . 19
M14 · Dataset, protocolo y benchmark reproducible . . . 20
M15 · Análisis científico, extensibilidad y rendimiento . . . 21
M16 · Documentación, release, memoria y defensa . . . 22
05 · Cómo organizar GitHub sin burocracia . . . 23
06 · Experimentos: cómo empezar sin esperar al final . . . 24
07 · Preparar la defensa desde cada módulo . . . 25
08 · Seguimiento semanal con tu mentor . . . 26
09 · Lista de comprobación final . . . 27
10 · Glosario mínimo para estudiar . . . 28
11 · Referencias, límites y siguiente acción . . . 29

Qué vas a encontrar: hoja de ruta, arranque práctico, 16 módulos, protocolo de IA, criterios de cierre, guía de experimentación, plantillas y plan de preparación de la defensa.

01 · El proyecto en cuatro hitos

Los módulos son áreas de responsabilidad, **no necesariamente carpetas, microservicios ni fases que debas recorrer una a una**. Los hitos siguientes permiten construir primero un recorrido visible y luego incorporar complejidad. El orden puede ajustarse tras auditar el estado real.

HITO A Navegación funcional sin IA

M01 + M02 + M07 + M08 + M09 + M10

Edificio sintético, espacios, grafo, cálculo de rutas y representación 2D. Prioriza un resultado navegable y comprobable.

HITO B Del plano a un modelo validado

M03 + M04 + M05 + M06 + M02/M07

Plano de entrada, extracción por método real, normalización, revisión humana y persistencia coherente.

HITO C Representación espacial completa

M11 + M12

Generación 3D determinista y sincronización de espacios y rutas entre 2D y 3D.

HITO D Evidencia y entrega académica

M14 + M15 + M16

Benchmark reproducible, análisis, memoria, repositorio publicable, ensayo de defensa y manuscrito.

> **Trabajo paralelo:** M13 (pruebas, calidad y seguridad) acompaña a todos los hitos; M14 empieza pronto con planos, permisos, métricas y protocolo; M16 se redacta progresivamente. No hay un «mes final para documentar».

Recorrido de datos que debes dominar

→ Plano -> preprocesamiento -> propuesta del extractor (IA o procedimental)

→ Propuesta -> esquema común -> validación -> corrección humana

→ Modelo aprobado -> PostGIS -> generación de grafo -> rutas

→ Mismo modelo -> GeoJSON/Leaflet y geometría 3D/Three.js

→ Ejecuciones + datos de referencia -> benchmark -> resultados -> memoria y defensa

02 · Tu punto de partida: seis sesiones

No se trata de seis días obligatorios. Cada sesión puede durar el tiempo que necesites. El objetivo es convertir la incertidumbre en información comprobada y terminar con una primera tarea pequeña.

Sesión 1 · Arrancar

Haz: Clona, levanta Docker, aplica migraciones, importa el ejemplo, prueba API y tests. Conserva comandos y errores.
Entrega: Entorno reproducible o lista de bloqueos verificables.

Sesión 2 · Dibujar

Haz: Sin IA, dibuja y explica plano -> proveedor -> propuesta -> revisión -> PostGIS -> grafo -> ruta -> UI.
Entrega: Mapa de flujo con huecos señalados.

Sesión 3 · Inventariar

Haz: Revisa README, modelos, servicios, pruebas y frontend. Clasifica: comprendido y probado / funciona pero no entiendo / incompleto.
Entrega: Inventario de capacidades con evidencias.

Sesión 4 · Elegir una frontera

Haz: Contrasta schema Pydantic, mock y ApprovalService: identifica diferencias de formato y dónde se pierde el resultado del proveedor.
Entrega: Una hipótesis técnica y un test que pueda falsarla.

Sesión 5 · Resolver una tarea

Haz: Con IA si ayuda, conecta correctamente una parte pequeña del flujo y crea la prueba que demuestra su comportamiento.
Entrega: Un cambio acotado, revisado y probado.

Sesión 6 · Explicar y planificar

Haz: Demuestra el cambio, explica alternativas y errores, registra una ADR breve si hay decisión relevante y define el siguiente issue.
Entrega: Primer hito demostrable y backlog de 2-3 tareas.

> **No hagas ahora:** borrar todo y recrearlo; estudiar cursos completos de cada framework; reorganizar por estética; o abrir 80 issues antes de probar que la primera tarea es viable.

03 · Protocolo de trabajo con IA

La IA puede generar la mayor parte del código, pero el criterio de aceptación no es «lo ha generado»: es **funciona, está verificado y puedes explicar sus decisiones relevantes.** Usa dos modos distintos según el momento.

Modo A · IA como tutor

Cuando una decisión no te resulte comprensible, pídele que formule preguntas y pequeños ejemplos, no una implementación completa. Debe ayudarte a explicar el modelo y anticipar errores.

> «Explícame este concepto aplicado a CampusNav3D. Primero pregúntame cómo lo entiendo, después corrige mis errores. No me des la solución completa hasta que haya intentado proponer una.»

Modo B · IA como implementador

Cuando el objetivo y los criterios de aceptación estén claros, permite generar código o pruebas. Tú revisas el diff, verificas la ejecución, corriges los límites y decides qué se integra.

> «Implementa solo el alcance del issue. Resume archivos tocados, decisiones, supuestos, pruebas ejecutadas y riesgos. Si el contrato actual no es coherente, detente y expón el conflicto antes de cambiar otros módulos.»

Ciclo mínimo para cada tarea

01. Formula el problema con palabras propias: entrada, salida, errores y criterio de aceptación.
02. Dibuja el flujo de datos o escribe un ejemplo pequeño y comprobable.
03. Consulta documentación y pide ayuda a IA; decide qué alternativa usar y por qué.
04. Integra cambios pequeños en una rama, revisa el diff y ejecuta pruebas relevantes.
05. Explica lo que cambió sin apoyarte en la IA y realiza una modificación pequeña o provoca un error controlado.
06. Documenta los resultados y cierra el issue solo cuando hay evidencia.

No es necesario memorizar cada API: sí debes dominar responsabilidades, entradas/salidas, condiciones de fallo, dependencias y decisiones de arquitectura. Consulta las normas de tu universidad sobre uso y declaración de IA.

04 · Los 16 módulos de CampusNav3D

Cada ficha contiene un objetivo, conocimientos que adquirir, acciones en secuencia, evidencias de cierre y preguntas que podrían aparecer en la defensa. Los estados iniciales se basan en el repositorio revisado: revísalos tú antes de ponerte a implementar.

Bloque	Módulos	Resultado
Fundamentos	M01, M02, M03	Entorno y datos espaciales consistentes
Extracción y validación	M04, M05, M06	Plano transformado en datos aprobables
Navegación	M07, M08	Grafo y rutas que respetan restricciones
Servicios	M09	API y trabajos asíncronos integrados
Visualización	M10, M11, M12	2D/3D coherentes y sincronizados
Transversales	M13, M14, M15, M16	Calidad, experimentos y defensa

> **Cómo avanzar:** al comenzar un módulo, crea como mucho los issues de la siguiente iteración. Al terminarlo, graba una demo corta, conserva pruebas y redacta una explicación de 10-15 líneas con tus propias palabras.

M01 · Entorno e infraestructura de desarrollo

FUNDAMENTOS · Guía: 3, 15, 17, 31 y fase 1

> **Resultado que debes conseguir.** Disponer de un entorno que tú y cualquier evaluador podáis levantar sin pasos ocultos, con pruebas automatizadas y sin secretos publicados.

Estado inicial observado: Base avanzada; verificar reproducibilidad. **Dependencias:** Inicio del proyecto.

Rutas de referencia: .github/workflows/ci.yml · docker-compose.yml · backend/core/settings.py.

Qué necesitas entender

- Qué aporta cada servicio: Django, PostGIS, Redis y Celery; qué significa que un contenedor esté sano.
- Diferencia entre configuración, dependencia y estado persistente; fundamentos de Git, ramas, commits y PR.
- Qué comprueba CI y qué no demuestra una ejecución verde.

Paso a paso

01. Clona el repositorio en una carpeta limpia. Lee el README y anota cada requisito necesario para arrancar el sistema.

02. Levanta Docker Compose; comprueba los servicios y registra los problemas encontrados, sin cambiar todavía la arquitectura.

03. Ejecuta migraciones, importa el edificio de ejemplo y consulta un endpoint. Guarda comandos y resultados esperados.

04. Ejecuta la suite de tests y revisa el workflow de GitHub Actions del commit actual. Clasifica los fallos por causa.

05. Comprueba que no se versionen credenciales o datos privados. Aclara los valores de .env.example y la diferencia entre desarrollo y producción.

06. Actualiza las instrucciones de instalación para que una segunda persona repita los pasos.

Cómo demostrar que está terminado

Un equipo limpio puede levantar el backend con instrucciones verificadas.

Migraciones, ejemplo de datos y pruebas se pueden repetir; los problemas conocidos constan por escrito.

Carlos sabe explicar la función de cada servicio sin leer un tutorial.

Preguntas de defensa

? ¿Por qué Celery y Redis, en vez de ejecutar todo dentro del request HTTP?

? ¿Qué demuestra y qué no demuestra tu pipeline de integración continua?

Ojo: No confundas «Docker levanta» con «el sistema resuelve el problema». La infraestructura es un medio.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M02 · Modelo de dominio y persistencia espacial

FUNDAMENTOS · Guía: 6, 7, 8 y FR-001

> **Resultado que debes conseguir.** Representar correctamente campus, edificios, plantas, espacios y conexiones mediante un modelo estable del que dependan la navegación y las visualizaciones.

Estado inicial observado: Núcleo presente; faltan entidades y CRS coherente. **Dependencias:** M01.

Rutas de referencia: backend/maps/models/ · backend/maps/migrations/ · geometry_utils.py.

Qué necesitas entender

- Entidades, relaciones, restricciones e identificadores; cómo opera un ORM sobre PostgreSQL.
- Diferencia entre GeoJSON, WKT y geometrías almacenadas en PostGIS.
- Qué significa SRID/CRS y cuándo necesitas coordenadas métricas frente a latitud/longitud.

Paso a paso

01. Dibuja el modelo actual sin consultar la IA: Campus > Building > Floor > Space y relaciones existentes.

02. Contrasta las entidades con la guía; identifica Portals y VerticalConnections aún no representados.

03. Define qué información debe tener cada entidad y qué restricciones evitan inconsistencias (planta válida, identificador estable, geometría correcta).

04. Decide y documenta un sistema de coordenadas para trabajo interior y su correspondencia con datos externos.

05. Implementa o adapta modelos y migraciones por cambios pequeños, conservando compatibilidad cuando proceda.

06. Prepara un edificio sintético con al menos dos plantas, espacios, puertas y conexión vertical; crea pruebas de integridad.

Cómo demostrar que está terminado

El dominio puede describir más de un edificio y sus plantas sin lógica especial hardcoded.

Las geometrías tienen unidades coherentes y validaciones; no se mezclan metros con grados.

Hay fixtures y tests sobre relaciones, restricciones e identificadores.

Preguntas de defensa

? ¿Por qué PostGIS y no una tabla con coordenadas JSON?

? ¿Qué diferencia hay entre un espacio, una puerta y un nodo del grafo?

? ¿Qué ocurre si cambias de sistema de coordenadas?

Ojo: El cálculo actual de longitudes sobre SRID 4326 no proporciona metros directamente.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M03 · Importación y preprocesamiento de planos

PROCESAMIENTO · Guía: 9 y FR-002

> **Resultado que debes conseguir.** Convertir un archivo de entrada en un recurso trazable y listo para el extractor, con escala, unidades y configuración explícitas.

Estado inicial observado: Subida parcial; transformación real pendiente. **Dependencias:** M01 · M02.

Rutas de referencia: backend/maps/services/plan_service.py · models/spatial.py.

Qué necesitas entender

- Formatos PDF, PNG y JPG, imágenes y resolución; tamaño, MIME real y hashing.
- Conceptos de escala, píxel, rotación, referencia espacial y transformaciones de coordenadas.
- Diferencia entre procesamiento síncrono y diferido.

Paso a paso

01. Define el contrato de entrada (formatos, tamaño, planta de destino, permisos de publicación).

02. Prueba la subida con archivos válidos, corruptos y duplicados; registra el comportamiento observado.

03. Implementa conversión de PDF a imagen y la selección de página si se requieren varios folios.

04. Normaliza orientación, resolución y escala; conserva siempre original, parámetros y hash.

05. Define una transformación verificable entre píxeles y coordenadas del modelo espacial.

06. Entrega al extractor un artefacto normalizado y añade pruebas con un plano sintético conocido.

Cómo demostrar que está terminado

Cada archivo genera un registro identificable, con procedencia y hash.

La misma entrada y parámetros producen una preparación trazable.

Los casos inválidos terminan con errores comprensibles y sin registros incoherentes.

Preguntas de defensa

? ¿Cómo pasas de una coordenada en píxeles a una dimensión física?

? ¿Cómo distingues dos subidas iguales de dos versiones diferentes?

Ojo: Una imagen sin escala conocida no permite afirmar medidas físicas exactas.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M04 · Extractores procedimentales y proveedores de IA

PROCESAMIENTO · Guía: 10 y FR-003/FR-004

> **Resultado que debes conseguir.** Obtener propuestas de espacios, accesos y etiquetas a través de proveedores intercambiables, manteniendo un método de referencia sin IA.

Estado inicial observado: Contrato y mock; proveedor real pendiente. **Dependencias:** M03 · M05.

Rutas de referencia: backend/maps/providers/ · services/ingestion_service.py.

Qué necesitas entender

- Interfaz/contrato frente a implementación; inversión de dependencias y patrón Adapter.
- Diferencias entre mock de pruebas, procedimiento determinista y modelo de IA real.
- Manejo de timeouts, respuestas inválidas, cuotas, costes, logs y reintentos.

Paso a paso

01. Escribe el contrato único de extracción: archivo, parámetros, versión del esquema, resultados y metadatos.

02. Construye un mock predecible para pruebas contractuales; no lo uses como línea base científica.

03. Diseña un extractor procedimental real, con supuestos y límites explicitados.

04. Integra al menos un proveedor de IA real mediante adaptador; guarda modelo, prompts, parámetros y coste.

05. Haz que el servicio de ingesta persista la propuesta devuelta por el proveedor, nunca un resultado fijo de ejemplo.

06. Prueba resultados normales, vacíos, mal formados, lentos y fallidos. Verifica que cambiar de proveedor no altere el núcleo.

Cómo demostrar que está terminado

Mock, procedimental y proveedor real satisfacen el mismo contrato.

La respuesta real del proveedor atraviesa el pipeline y puede ser auditada.

Los fallos se registran y no producen datos aprobados por accidente.

Preguntas de defensa

? ¿Por qué un mock no demuestra que el extractor es correcto?

? ¿Qué partes deben mantenerse iguales al sustituir el proveedor?

? ¿Qué limitaciones tiene una salida de IA aparentemente válida?

Ojo: El repositorio, en la revisión, llamaba al extractor pero guardaba geometrías fijas: resolver antes de interpretar resultados.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M05 · Normalización, geometrías y validación espacial

PROCESAMIENTO · Guía: 9.4, 9.5 y FR-006

> **Resultado que debes conseguir.** Garantizar que cualquier propuesta entra en el dominio mediante un esquema único y supera controles geométricos y topológicos antes de aprobarse.

Estado inicial observado: Esquema existente; contratos incompatibles. **Dependencias:** M02 · M03 · M04.

Rutas de referencia: backend/maps/schemas.py · geometry_utils.py · services/approval_service.py.

Qué necesitas entender

- Esquemas Pydantic, validación de tipos, semántica y versiones.
- Polígonos válidos, anillos cerrados, intersecciones, adyacencias y grafo de referencia.
- Diferencia entre validar estructura y validar significado espacial.

Paso a paso

01. Contrasta el esquema Pydantic actual con lo que espera ApprovalService: puntos {x,y} vs listas [x,y], y space_type vs type.

02. Elige un formato intermedio canónico y documenta versión, unidades, sistemas de referencia e identificadores.

03. Implementa normalizadores por proveedor, sin trasladar estructuras privadas al dominio.

04. Añade validaciones para polígonos autocruzados, espacios fuera de límites, duplicados y conexiones imposibles.

05. Comprueba que los errores de validación se muestran en revisión humana y bloquean la publicación.

06. Crea fixtures de casos correctos e incorrectos, con tests de aceptación y regresión.

Cómo demostrar que está terminado

Toda propuesta aceptable cumple un único contrato versionado.

Se impide publicar un modelo geométrica o topológicamente inválido.

Los tests cubren casos difíciles, no solo geometrías ideales.

Preguntas de defensa

? ¿Qué valida Pydantic y qué debería validar PostGIS o la lógica de dominio?

? ¿Cuándo una geometría formalmente válida sigue siendo incorrecta?

Ojo: La consistencia del formato entre extracción y aprobación es un bloqueo prioritario.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M06 · Corrección humana, estados y versionado

PROCESAMIENTO · Guía: 9.2, 11 y FR-005/FR-012

> **Resultado que debes conseguir.** Permitir que una persona inspeccione y corrija las propuestas de IA, y conservar una historia verificable hasta publicar el modelo.

Estado inicial observado: Aprobación/rechazo por API; editor pendiente. **Dependencias:** M04 · M05 · M09 · M10.

Rutas de referencia: backend/maps/services/approval_service.py · models/spatial.py.

Qué necesitas entender

- Máquinas de estados y transiciones válidas; transacciones ACID.
- Revisión human-in-the-loop, eventos de auditoría y versionado.
- Diferencia entre borrador, modelo validado y modelo publicado.

Paso a paso

01. Dibuja el ciclo completo del plano y las transiciones válidas; compara con los estados actuales del código.

02. Evita transiciones arbitrarias y define qué operaciones puede realizar cada rol.

03. Diseña una pantalla mínima para superponer el plano y editar polígonos, puertas, etiquetas y relaciones.

04. Registra cambios con valor anterior, valor nuevo, usuario, fecha y ejecución de origen.

05. Implementa aprobación transaccional y rechazo sin datos parciales; permite recuperar una versión previa.

06. Prueba reintentos, rechazo, errores concurrentes y actualización de vistas tras aprobar.

Cómo demostrar que está terminado

Se puede revisar, corregir y aprobar un plano sin alterar directamente datos definitivos.

Cada cambio importante queda trazado y puede explicarse.

La aprobación crea entidades espaciales coherentes y no pierde el historial.

Preguntas de defensa

? ¿Por qué la IA no es la fuente de verdad final?

? ¿Qué asegura una transacción si falla la creación de la cuarta habitación?

Ojo: No confundir un endpoint de aprobación con un editor humano completo.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M07 · Generación y validación del grafo de navegación

NAVEGACIÓN · Guía: 12.1-12.3 y FR-007

> **Resultado que debes conseguir.** Construir un grafo correcto a partir del modelo espacial validado y verificar que representa conexiones físicamente transitables.

Estado inicial observado: Grafo desde aristas preexistentes; derivación incompleta. **Dependencias:** M02 · M05 · M06.

Rutas de referencia: backend/maps/graph_builder.py · models/spatial.py.

Qué necesitas entender

- Grafo dirigido, nodo, arista, grado, componentes conexas y topología.
- Cómo se relacionan Space, Portal y VerticalConnection con nodos y aristas.
- Diferencia entre una conexión declarada y una conexión deducida y validada.

Paso a paso

01. Explica cómo GraphBuilder actual construye nx.DiGraph a partir de Space y NavigationEdge.

02. Dibuja un edificio pequeño con cinco espacios y una puerta bloqueada; identifica sus nodos y aristas.

03. Define reglas para derivar conexiones desde puertas/pasillos/escaleras, no solo desde aristas creadas a mano.

04. Construye el grafo desde datos validados; conserva identificadores que enlacen con GeoJSON y 3D.

05. Detecta espacios navegables aislados, conexiones atravesando muros y errores entre plantas.

06. Prueba grafos con bucles, rutas imposibles, cambios de planta y conexiones restringidas.

Cómo demostrar que está terminado

Un edificio validado produce un grafo útil sin insertar manualmente cada arista.

Los espacios sin conexión válida quedan señalados.

Las pruebas detectan inconsistencias espaciales y topológicas.

Preguntas de defensa

? ¿Por qué usar un grafo dirigido?

? ¿Qué representa realmente cada nodo y cada arista?

? ¿Qué pasa si hay dos puertas entre los mismos espacios?

Ojo: Que NetworkX calcule un camino no demuestra que el camino exista físicamente.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M08 · Motor de rutas y perfiles de accesibilidad

NAVEGACIÓN · Guía: 12.4-12.5, 24, FR-008/FR-009

> **Resultado que debes conseguir.** Calcular rutas que respeten distancia, direccionalidad, restricciones y necesidades de accesibilidad.

Estado inicial observado: Estrategias existentes; pesos y accesibilidad incompletos. **Dependencias:** M07.

Rutas de referencia: backend/maps/routing_strategies.py · navigation_facade.py.

Qué necesitas entender

- Dijkstra, camino mínimo, costes de arista y grafos ponderados.
- Diferencia entre ruta más corta, más rápida y accesible.
- Métricas de validez, latencia, conectividad y optimalidad.

Paso a paso

01. Revisa las estrategias fastest y accessible actuales; define sus diferencias reales de coste y restricciones.

02. Corrige el uso de unidades: distancia interior expresada de forma coherente, preferentemente en metros.

03. Define perfiles de ruta, incluidas restricciones de escaleras, ascensores y zonas no permitidas.

04. Implementa cambios de planta y una respuesta de ruta con segmentos, unidades y advertencias claras.

05. Añade pruebas de rutas imposibles, aristas cerradas, destinos inaccesibles y preferencias conflictivas.

06. Evalúa pares origen-destino y registra rutas válidas, tiempos y violaciones de restricciones.

Cómo demostrar que está terminado

Ninguna ruta válida atraviesa elementos no transitables según el modelo.

Las restricciones de accesibilidad se aplican y se pueden demostrar.

Las distancias y tiempos tienen unidades y origen justificables.

Preguntas de defensa

? ¿Qué optimiza Dijkstra y de dónde salen los pesos?

? ¿Cuándo una ruta más corta no es la más rápida?

? ¿Cómo compruebas que una ruta accesible realmente lo es?

Ojo: No presentar como «ruta accesible» un filtro genérico de aristas que todavía no representa escaleras y ascensores correctamente.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M09 · API, orquestación y trabajos asíncronos

SERVICIOS · Guía: 14, FR-011 y NFR-009

> **Resultado que debes conseguir.** Exponer casos de uso del sistema mediante una API estable y coordinar los trabajos sin mezclar HTTP, lógica de dominio y ejecución pesada.

Estado inicial observado: Base avanzada; integración extremo a extremo pendiente. **Dependencias:** M02-M08.

Rutas de referencia: backend/maps/views.py · services/ · tasks.py · urls.py.

Qué necesitas entender

- HTTP y REST: códigos, autenticación, errores y contratos OpenAPI.
- Diferencia entre vista, servicio, tarea y operación de base de datos.
- Reintentos, idempotencia, concurrencia, transacciones y observabilidad.

Paso a paso

01. Enumera los casos de uso principales y asócialos a endpoints y servicios existentes.

02. Traza la ruta real de una petición de upload hasta Celery y la respuesta de estado.

03. Conecta salida del extractor, validación, aprobación y generación posterior del grafo sin saltos ni datos de prueba ocultos.

04. Define manejo de timeouts, reintentos seguros y errores de usuario frente a errores internos.

05. Documenta API en OpenAPI; añade pruebas de integración y permisos.

06. Ejecuta un flujo verificable: plano -> propuesta -> revisión -> persistencia -> ruta.

Cómo demostrar que está terminado

El backend proporciona un recorrido completo con resultados reales del proceso.

Cada fallo devuelve estado apropiado, mensaje interpretable y registro trazable.

Los servicios de dominio no dependen directamente del framework de UI.

Preguntas de defensa

? ¿Por qué un trabajo largo se devuelve con 202 Accepted?

? ¿Cómo evitas que un reintento de Celery duplique espacios?

Ojo: Un endpoint que responde 200 no implica que el flujo científico esté terminado.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M10 · Aplicación web y mapa 2D

VISUALIZACIÓN · Guía: 11, 13.1 y FR-010

> **Resultado que debes conseguir.** Hacer visible y operable el dominio: elegir edificio/planta, localizar espacios, calcular rutas y revisar resultados de extracción.

Estado inicial observado: React inicial; Leaflet pendiente. **Dependencias:** M02 · M08 · M09.

Rutas de referencia: frontend/src/App.jsx · Admin.jsx · frontend/package.json.

Qué necesitas entender

- Componentes React, estado, efectos y clientes API.
- Leaflet, capas, geometrías GeoJSON, referencias espaciales y selección.
- Conceptos básicos de UI accesible y estados de carga/error.

Paso a paso

01. Ejecuta el frontend actual y verifica el login, el listado de campus y la API consumida.

02. Define las pantallas mínimas: mapa público, selección origen/destino y revisión administrativa.

03. Añade Leaflet y muestra un GeoJSON con espacios identificables.

04. Permite cambiar planta y marcar origen/destino; pinta la ruta del backend y explica sus unidades.

05. Añade estados de carga, errores, ausencia de ruta y accesibilidad básica de controles.

06. Conecta el editor del M06 a la vista administrativa, con feedback de validaciones.

Cómo demostrar que está terminado

Un usuario puede seleccionar un edificio/planta y ver espacios y rutas reales.

La UI conserva identificadores del dominio y gestiona errores.

El panel de revisión permite demostrar el paso humano de validación.

Preguntas de defensa

? ¿Por qué GeoJSON es apropiado para la representación 2D?

? ¿Qué estados necesita la UI cuando la extracción es asíncrona?

Ojo: El repositorio aún solo ofrece listado público y panel de administración provisional.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M11 · Generación y visualización 3D

VISUALIZACIÓN · Guía: 13.2 y 13.4

> **Resultado que debes conseguir.** Crear vistas 3D coherentes y repetibles a partir del mismo modelo espacial validado, sin inventar libremente los edificios.

Estado inicial observado: Pendiente. **Dependencias:** M02 · M05 · M10.

Rutas de referencia: frontend/ · futuro generador/escena Three.js.

Qué necesitas entender

- Extrusión de polígonos, altura, coordenadas 3D, mallas y escena.
- Three.js: geometría, materiales, cámara, interacción y rendimiento.
- Diferencia entre fidelidad visual y fidelidad geométrica.

Paso a paso

01. Fija un caso sintético con geometrías conocidas y alturas documentadas.

02. Define un contrato de datos 3D que use los identificadores de Space y las unidades del modelo.

03. Extruye suelos y paredes; añade puertas y estructura mínima de plantas.

04. Renderiza una escena Three.js básica y permite seleccionar habitaciones.

05. Implementa cambios de planta y visualización de ruta si el modelo lo permite.

06. Mide tiempos de carga y tamaño de escena; optimiza solo con evidencia de necesidad.

Cómo demostrar que está terminado

La misma entrada validada genera una escena 3D reproducible.

Cada objeto tiene identificador estable para enlazarlo con las otras representaciones.

Las dimensiones y límites de precisión están documentados.

Preguntas de defensa

? ¿Por qué generar 3D de forma determinista y no con una IA generativa libre?

? ¿Cómo garantizas que un espacio 3D corresponde al espacio persistido?

Ojo: No presentar un render visualmente atractivo como evidencia de precisión espacial.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M12 · Sincronización entre 2D, 3D y navegación

VISUALIZACIÓN · Guía: 13.3 y FR-010

> **Resultado que debes conseguir.** Garantizar que las vistas, la ruta y la base de datos representan la misma entidad y reaccionan de forma coherente a las acciones del usuario.

Estado inicial observado: Pendiente. **Dependencias:** M08 · M10 · M11.

Rutas de referencia: frontend/ · API y modelo de identificadores.

Qué necesitas entender

- Estado compartido y flujo unidireccional de datos.
- Identificadores estables, cambios de versión y eventos de selección.
- Pruebas de sincronización y regresión entre representaciones.

Paso a paso

01. Define el estado canónico: edificio, planta, espacio seleccionado, ruta y versión del modelo.

02. Unifica identificadores para PostGIS, GeoJSON, grafo, objetos 3D y API.

03. Implementa selección en 2D que resalte el mismo espacio 3D.

04. Implementa selección en 3D que resalte el elemento 2D; sincroniza cambios de planta.

05. Pinta la misma ruta en ambas vistas y verifica que no cambia su significado.

06. Tras aprobar una corrección, regenera datos y comprueba sincronización y compatibilidad de IDs.

Cómo demostrar que está terminado

Selección bidireccional y rutas funcionan con identificadores comunes.

Las vistas se actualizan sin inconsistencias al cambiar de planta o versión.

Existe al menos una prueba E2E por escenario crítico de sincronización.

Preguntas de defensa

? ¿Dónde resides el estado que comparten ambas vistas?

? ¿Cómo evitas que modificar un plano produzca objetos 3D desincronizados?

Ojo: Dos pantallas que muestran edificios similares no son necesariamente vistas sincronizadas.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M13 · Calidad, pruebas, seguridad y accesibilidad

TRANSVERSAL · Guía: 17, 18, 31, 32, 33 y 41

> **Resultado que debes conseguir.** Demostrar que el comportamiento relevante es correcto, repetible, seguro y utilizable; impedir que nuevos cambios rompan capacidades anteriores.

Estado inicial observado: Suite y CI existentes; cobertura por completar. **Dependencias:** Atraviesa todos los módulos.

Rutas de referencia: backend/maps/tests/ · .github/workflows/ci.yml.

Qué necesitas entender

- Prueba unitaria, integración, contrato, E2E, regresión y rendimiento.
- Falsos positivos de tests y diferencia entre cobertura de código y cobertura funcional.
- Permisos, secretos, planos sensibles, condiciones de despliegue y WCAG 2.2.

Paso a paso

01. Define una matriz de pruebas por requisito funcional, no solo por archivo de código.

02. Asegura pruebas unitarias del dominio, geometrías, grafos, rutas y normalizadores.

03. Añade integración entre PostGIS, API, Celery y un proveedor simulado.

04. Construye E2E sobre el recorrido completo y las dos visualizaciones, cuando existan.

05. Verifica CI, migraciones, permisos, secretos y ausencia de planos no autorizados.

06. Registra limitaciones conocidas y ejecuta regresiones antes de cerrar cada módulo.

Cómo demostrar que está terminado

Cada funcionalidad crítica tiene al menos una evidencia automatizada o manual reproducible.

Los tests no dependen de geometrías hardcoded que oculten el fallo de integración.

El repositorio no contiene secretos ni datos institucionales publicados sin autorización.

Preguntas de defensa

? ¿Qué riesgo no cubren tus tests actuales?

? ¿Por qué CI en verde no implica que todo el sistema funcione?

Ojo: No aumentar el número de tests sin comprobar que validan el comportamiento correcto.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M14 · Dataset, protocolo y benchmark reproducible

INVESTIGACIÓN · Guía: 19-24 y 29

> **Resultado que debes conseguir.** Obtener evidencias comparables de tiempo, coste, exactitud y corrección humana para evaluar la aportación de la IA sin inventar métricas.

Estado inicial observado: Pendiente; empezar a planificar pronto. **Dependencias:** M03-M09 y M13, en paralelo.

Rutas de referencia: benchmark/ · test_img/ · futuros scripts experimentales.

Qué necesitas entender

- Preguntas RQ1-RQ3 y condiciones B0 manual, B1 procedimental, B2 IA, B3 segundo proveedor.
- Anotaciones de referencia, IoU, precision, recall, F1, tiempos y fallos.
- Reproducibilidad experimental, condiciones de comparación y sesgos.

Paso a paso

01. Define preguntas de investigación y qué variable mide cada una.

02. Consigue planos autorizados o sintéticos y crea anotaciones de referencia fiables. La guía recomienda un mínimo defendible de 12 planos, 15-20 preferibles y varios edificios.

03. Diseña B0/B1/B2 sobre las mismas entradas, con igual criterio de salida validado; define B3 si procede.

04. Define cómo medir por separado preparación, procesamiento automático, corrección humana, espera y coste.

05. Congela versiones de datos, código, prompts, proveedor y parámetros antes de las ejecuciones finales.

06. Ejecuta scripts de benchmark, conserva resultados brutos y documenta fallos; genera tablas reproducibles.

Cómo demostrar que está terminado

El dataset tiene procedencia y anotaciones documentadas.

Otra persona puede ejecutar los experimentos y obtener tablas verificables.

Las afirmaciones de ahorro o calidad se apoyan en datos observados y en comparaciones justas.

Preguntas de defensa

? ¿Por qué necesitas comparar con un proceso procedimental sin IA?

? ¿Qué mide IoU y qué información importante no captura?

? ¿Cómo incluyes el tiempo de corrección humana en la comparación?

Ojo: El mock actual no sirve como condición B1: no analiza los planos de entrada.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M15 · Análisis científico, extensibilidad y rendimiento

INVESTIGACIÓN · Guía: 25-28 y 43

> **Resultado que debes conseguir.** Interpretar resultados con rigor, demostrar extensibilidad mediante cambios reales y reconocer amenazas a la validez.

Estado inicial observado: Pendiente. **Dependencias:** M14 · pruebas de módulos completados.

Rutas de referencia: benchmark/analysis/ · documentación de arquitectura.

Qué necesitas entender

- Estadística descriptiva y comparaciones pareadas; unidad de análisis y dependencia entre observaciones.
- Escenarios de cambio: proveedor, algoritmo de rutas, tipo de espacio, formato y generador 3D.
- Latencia p50/p95, FPS, coste, tamaño de entrada y amenazas a la validez.

Paso a paso

01. Define la unidad de análisis por métrica: plano, espacio, puerta, ruta o ejecución.

02. Resume resultados con tamaños de muestra, mediana, dispersión y errores, sin ocultar fallos.

03. Realiza análisis comparativos apropiados y explica cuándo la muestra no soporta generalizaciones.

04. Ejecuta escenarios de extensibilidad reales; registra archivos modificados, tiempo y regresiones.

05. Mide el rendimiento de API, enrutamiento, tareas y frontend en entornos documentados.

06. Redacta límites, fuentes de sesgo y amenazas internas, externas, de constructo y estadísticas.

Cómo demostrar que está terminado

Toda conclusión cuantitativa puede trazarse a resultados brutos y scripts.

La extensibilidad se evidencia con cambios observables, no con afirmaciones absolutas.

El análisis reconoce limitaciones y no vende una precisión no medida.

Preguntas de defensa

? ¿Qué resultados respaldan que cambiar de proveedor es fácil?

? ¿Puedes generalizar de unas plantas sintéticas a cualquier universidad?

? ¿Qué observaciones son independientes en tu análisis?

Ojo: No usar líneas de código o estrellas de GitHub como prueba única de extensibilidad o impacto.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

M16 · Documentación, release, memoria y defensa

INVESTIGACIÓN · Guía: 29, 30, 35-36 y 40-50

> **Resultado que debes conseguir.** Entregar un TFG comprensible y reproducible, preparar la disponibilidad del software y construir una defensa que Carlos pueda realizar con criterio propio.

Estado inicial observado: README y licencia; manuscrito y defensa pendientes. **Dependencias:** Desde el inicio; cierre al final.

Rutas de referencia: README.md · LICENSE · docs/ · CITATION.cff (pendiente).

Qué necesitas entender

- Cómo narrar motivación, contribución, arquitectura y resultados.
- Trazabilidad entre requisitos, decisiones, commits, métricas y memoria.
- Requisitos editoriales y de reproducibilidad; comunicación técnica y limitaciones.

Paso a paso

01. Mantén README y documentación de decisiones actualizados en cada hito, no solo al terminar.

02. Escribe la memoria progresivamente: problema, estado del arte, requisitos, diseño, implementación, pruebas y evaluación.

03. Documenta instalación limpia, ejemplos, API, modelos, proveedores, benchmarks, licencias y privacidad.

04. Antes de cerrar, crea release versionada y CITATION.cff; prepara DOI si corresponde y se autoriza la publicación.

05. Prepara el manuscrito de SoftwareX conforme a la plantilla oficial vigente; no inventes cifras y separa requisitos editoriales de criterios académicos.

06. Ensaya la defensa: problema -> arquitectura -> demo -> decisiones -> pruebas -> resultados -> límites; responde preguntas sin depender de la IA.

Cómo demostrar que está terminado

Una persona externa puede instalar, ejecutar y comprender el sistema con la documentación.

La memoria explica cómo se obtuvo cada resultado y reconoce sus límites.

Carlos puede defender decisiones, fallos y alternativas, y demostrar un recorrido completo.

Preguntas de defensa

? ¿Cuál es la contribución propia de tu TFG frente al uso de frameworks conocidos?

? ¿Qué parte del trabajo demuestra que la IA añade valor?

? ¿Qué cambiarías si tuvieras más tiempo?

Ojo: No esperar al último mes para escribir la memoria ni para estudiar la solución.

MI REGISTRO · □ Por hacer □ En curso □ Probado □ Comprendido

Evidencia / PR: .....

Siguiente duda o paso: .....

05 · Cómo organizar GitHub sin burocracia

Los 16 módulos son un **mapa maestro**. En GitHub, usa cuatro milestones para los hitos A-D, una issue de referencia por módulo si resulta útil y tareas pequeñas para el trabajo diario. No necesitas un tablero empresarial complejo ni ceremonias de Scrum completas.

Tablero de trabajo

Columna	Qué significa
Backlog	Trabajo pendiente, no comprometido.
Esta semana	1-3 tareas elegidas y suficientemente definidas.
En curso	Máximo 1-2 tareas simultáneas.
En revisión	Tests, demo, explicación y revisión de diff.
Hecho	Pruebas y evidencia aceptadas, no solo código generado.

Plantilla de issue lista para copiar

> **Título:** [M05] Normalizar la propuesta de extracción antes de aprobar.
> **Problema:** el extractor devuelve {x, y}/space_type y la aprobación espera [x, y]/type.
> **Alcance:** adaptar el contrato y el flujo; no rediseñar toda la arquitectura.
> **Aceptación:** la propuesta real se valida, persiste y el test falla si se vuelve al formato anterior.
> **Evidencia:** tests ejecutados, diff, ejemplo de entrada/salida.
> **Aprendizaje:** explicar dónde se valida el esquema y qué ocurre si falla.

Criterio de «Hecho» para una tarea

- Cumple sus criterios de aceptación con una demo o una prueba verificable.
- Comprueba el caso normal, un caso de error y, si afecta a datos, la integridad de persistencia.
- El cambio está aislado y versionado; no hay secretos ni datos sensibles.
- README o decisión técnica actualizados cuando procede.
- Puedes explicar el flujo, las razones y los fallos que podría sufrir.

06 · Experimentos: cómo empezar sin esperar al final

SoftwareX exige que el valor científico se sostenga en evidencia. La guía del tutor propone las preguntas RQ1-RQ3 y compara B0 (manual), B1 (procedimental sin IA) y B2 (asistido por IA); B3 permite explorar otro modelo/proveedor. No presupongas que B2 es mejor.

Condición	Qué hace	Qué debe incluir la medida
B0 · Manual	Una persona interpreta y vectoriza el plano.	Tiempo humano completo, correcciones y calidad final.
B1 · Procedimental	Algoritmo sin IA, con preprocesamiento declarado.	Preparación, tiempo automático y corrección humana.
B2 · IA	Extracción IA + validación + corrección.	Tokens/coste, tiempo de espera, errores y tiempo humano.
B3 · Modelo alternativo	Otro proveedor con el mismo contrato.	Mismos planos, salidas y métricas, con versión del modelo.

Protocolo experimental mínimo

01. Reúne planos utilizables legalmente; clasifícalos por dificultad, resolución, calidad de etiqueta y número de espacios.
02. Anota un «ground truth» razonable: espacios, puertas, conexiones y etiquetas; revisa errores de anotación.
03. Fija antes de medir las métricas: IoU por polígono, F1 de puertas/adyacencias, tiempo humano total y tasa de rutas válidas.
04. Usa las mismas entradas en las condiciones comparables; conserva también los fallos y reintentos.
05. Registra commit, versión, modelo, parámetros, seed si existe, sistema operativo, tiempo y coste.
06. Guarda resultados brutos CSV/JSON, añade un script que regenere tablas, y explica los límites de la muestra.

La guía indica como referencia experimental **12 planos como mínimo defendible**, 15-20 recomendables y tres edificios. Si no alcanzas la muestra, consúltalo con el tutor antes de cambiar el protocolo; no informes un estudio pequeño como si demostrara generalidad.

07 · Preparar la defensa desde cada módulo

No dejes el aprendizaje para el último mes. Al terminar cada módulo, prepara una ficha de comprensión de una página que puedas explicar en tres minutos. Es la mejor inversión contra la sensación de estar trabajando sobre código que no controlas.

Ficha de comprensión por módulo

1. **Problema:** ¿qué necesidad cubre este módulo?
2. **Entrada / salida:** ¿qué datos recibe y qué produce?
3. **Decisiones:** ¿por qué elegiste esta herramienta o abstracción?
4. **Dependencias:** ¿qué otros módulos utiliza y quién lo utiliza?
5. **Pruebas:** ¿cómo sabes que hace lo que afirmas?
6. **Errores:** ¿qué falla y cómo se detecta?
7. **Alternativas:** ¿qué otra solución viable había?
8. **Limitaciones:** ¿qué no resuelve todavía?

Ensayo progresivo de defensa

01. Cada semana, explica un módulo sin abrir un chatbot ni leer la implementación línea a línea.
02. El mentor formula dos preguntas de diseño y una de fallo hipotético; responde con el código, las pruebas o el diagrama como apoyo.
03. Una vez al mes, realiza una demo de 10 minutos de un recorrido de extremo a extremo y reconoce lo que no funciona.
04. Antes del cierre, ensaya una defensa completa con: problema, aportación, metodología, arquitectura, demo, resultados y límites.
05. Prepara respuestas específicas sobre la participación de la IA y sobre qué trabajo de verificación has hecho personalmente.

Mensaje clave: no necesitas recitar las librerías de memoria. Sí debes poder explicar por qué el modelo espacial es la fuente de verdad, cómo se calculan rutas y cómo se midió el valor añadido de la IA.

08 · Seguimiento semanal con tu mentor

Una reunión de 30-45 minutos basta si hay una demo y una tarea siguiente bien definida. Carlos es autor, desarrollador y responsable académico. El mentor ayuda a priorizar, razonar, revisar y detectar riesgos; no escribe ni presenta el trabajo por él.

Tiempo	Dinámica	Pregunta guía
10 min	Demo real	¿Qué funciona ahora que antes no funcionaba?
10-15 min	Revisión técnica	¿Qué decisiones tomaste y qué cambiarías?
10 min	Bloqueos y aprendizaje	¿Qué concepto todavía no dominas?
5-10 min	Siguiente microhito	¿Qué evidencia esperamos ver en la próxima sesión?

Minuta mínima después de la reunión

> **Hecho:** .....
> **Evidencia:** .....
> **Comprendido / pendiente:** .....
> **Riesgo principal:** .....
> **Próxima tarea:** .....
> **Decisión de arquitectura (si existe):** .....

Para evitar dispersarte: mantén un único siguiente paso claro. Si aparece un problema nuevo, apúntalo en el backlog y vuelve a la tarea activa, salvo que sea un bloqueo real o un defecto crítico.

09 · Lista de comprobación final

Esta lista resume los entregables que deberías revisar con el tutor antes del cierre. No sustituye la guía oficial, que sigue siendo la referencia de alcance. Los elementos recomendables y opcionales deben interpretarse según el acuerdo académico.

Producto y arquitectura

- ☐ Modelo espacial generalizable e identificadores compartidos.
- ☐ Pipeline de importación, IA/procedimental, validación y aprobación.
- ☐ Grafo generado y rutas verificadas; accesibilidad según requisitos.
- ☐ Mapas 2D y 3D sincronizados; editor de correcciones.
- ☐ API y servicios reproducibles; fallos y reintentos documentados.

Calidad y datos

- ☐ Tests unitarios, integración y E2E sobre flujos críticos.
- ☐ CI en estado conocido; documentación de instalación reproducible.
- ☐ Dataset con procedencia y permisos de uso.
- ☐ Sin credenciales ni planos sensibles publicados sin autorización.
- ☐ Métricas con resultados brutos y scripts reproducibles.

Investigación y publicación

- ☐ Comparaciones experimentales definidas y analizadas.
- ☐ Escenarios de extensibilidad con evidencia.
- ☐ Limitaciones, amenazas a la validez y fallos documentados.
- ☐ README, licencia, esquema API, memoria y documentación de uso.
- ☐ Release, referencia citable y manuscrito según las instrucciones vigentes.

Defensa y autoría

- ☐ Puedes explicar todas las decisiones arquitectónicas esenciales.
- ☐ Sabes qué partes provienen de IA y cómo las has verificado.
- ☐ Puedes mostrar un flujo completo y un fallo tratado correctamente.
- ☐ Cada conclusión de la memoria se conecta con código, pruebas o datos.

10 · Glosario mínimo para estudiar

Modelo de dominio. Representación estructurada de las entidades y reglas del problema, independiente de la interfaz.

GeoJSON. Formato JSON para geometrías y propiedades espaciales; suele utilizarse para intercambio y vistas.

PostGIS. Extensión espacial de PostgreSQL: geometrías, índices y consultas geográficas/geométricas.

CRS / SRID. Sistema de referencia que determina cómo interpretar las coordenadas y sus unidades.

Topología. Relaciones de conexión, adyacencia y continuidad entre espacios.

Grafo. Estructura de nodos y aristas que representa posibilidades de navegación.

Dijkstra. Algoritmo clásico para encontrar caminos de coste mínimo con pesos no negativos.

Celery / Redis. Ejecución diferida de trabajos y broker de mensajes, respectivamente.

Adapter. Abstracción que permite conectar proveedores con contratos distintos.

Contrato de datos. Esquema común que define entradas, salidas, tipos, semántica y validaciones.

Mock. Doble de prueba con comportamiento controlado; no equivale a un extractor real.

Baseline. Método de referencia contra el que comparar una propuesta experimental.

IoU / F1. Métricas de solapamiento de geometrías y de calidad de detección/clasificación.

Reproducibilidad. Capacidad de repetir un experimento siguiendo procedimiento, datos y versiones documentados.

Amenaza a la validez. Factor que limita o pone en duda la interpretación de los resultados.

Definition of Done. Condiciones explícitas y verificables que permiten dar por concluida una tarea.

11 · Referencias, límites y siguiente acción

Fuentes del manual

- **Guía del tutor:** *Guía de implementación y evaluación para convertir el TFG en un artículo publicable en SoftwareX*, proyecto CampusNav3D, versión 1.0, junio de 2026.
- **Repositorio:** github.com/DiZaster2002/CampusNav3D, archivos públicos de la rama principal revisados en noviembre de 2026.
- **Convenciones propuestas:** 16 módulos, hitos, fichas y método didáctico elaborados como ayuda de ejecución y aprendizaje; no son una clasificación oficial de la universidad.

Cuestiones que debes confirmar

- Fecha de entrega y carga de trabajo disponible: no están reflejadas en esta guía.
- Qué resultados y condiciones experimentales considera obligatorios el tutor para la evaluación académica.
- Normativa de tu universidad sobre IA, autoría, datos, privacidad y publicación de planos.
- Instrucciones editoriales vigentes de SoftwareX antes de preparar el envío.

> **Tu siguiente paso, Carlos:** abre el repositorio, levanta la aplicación y construye tu propio diagrama del recorrido «plano -> propuesta -> modelo validado -> grafo -> ruta». No borres nada aún. Marca qué etapas funcionan realmente, cuáles no y cuáles todavía no puedes explicar. Con eso tendrás la primera sesión de trabajo preparada.

La meta no es terminar con un repositorio enorme. Es terminar con una solución de la que puedas decir: **sé qué problema resuelve, por qué funciona, cómo lo he comprobado y cuáles son sus límites**.